

Міністерство освіти і науки України
Національний Університет “Львівська Політехніка”



Лабораторна робота
з дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав:
студент гр. ІХ-22
Пристайко М.

Прийняв:
Гордійчук-Бублівська О.В

Лабораторна робота 10

Тема: Модульне тестування об'єктно-орієнтованих програм у Python

Мета: Навчитися створювати модульні тести для класів і функцій у Python з використанням бібліотеки unittest, у тому числі з використанням параметризованих тестів і мок-об'єктів.

Завдання:

```
import unittest
from unittest.mock import Mock
from parameterized import parameterized
```

```
class MathTool:
    def add(self, a, b): return a + b
    def subtract(self, a, b): return a - b
    def multiply(self, a, b): return a * b
    def divide(self, a, b):
        if b == 0: raise ValueError("Ділення на нуль неможливе!")
        return a / b
```

```
class TestMathTool(unittest.TestCase):
    def setUp(self): self.math = MathTool()
    def tearDown(self): del self.math
    def test_add(self): self.assertEqual(self.math.add(3, 5), 8)
    def test_subtract(self): self.assertEqual(self.math.subtract(10, 4), 6)
    def test_multiply(self): self.assertEqual(self.math.multiply(3, 4), 12)
    def test_divide(self): self.assertEqual(self.math.divide(10, 2), 5)
    def test_divide_by_zero(self):
        with self.assertRaises(ValueError): self.math.divide(1, 0)
```

```
class LibraryItem:
    def __init__(self, title, author, year):
        self.title, self.author, self.year = title, author, year
    def details(self):
        return f'Назва: {self.title}, Автор: {self.author}, Рік: {self.year}'
```

```
class TestLibraryItem(unittest.TestCase):
    def setUp(self):
        self.book = LibraryItem("Кобзар", "Тарас Шевченко", 1840)
    def test_details(self):
        self.assertIn("Кобзар", self.book.details())
        self.assertIn("Тарас Шевченко", self.book.details())
        self.assertIn("1840", self.book.details())
```

```
class NotificationService:
    def send(self, user, message): return True
```

```
class UserManager:
```

```
def __init__(self, service): self.service = service
def notify_user(self, user, message):
    if self.service.send(user, message):
        return f"Повідомлення для {user} надіслано успішно"
    return f"Не вдалося надіслати повідомлення для {user}"
```

```
class TestUserManager(unittest.TestCase):
    def setUp(self):
        self.mock_service = Mock(spec=NotificationService)
        self.manager = UserManager(self.mock_service)
    def test_notify_success(self):
        self.mock_service.send.return_value = True
        self.assertEqual(
            self.manager.notify_user("alice", "Привіт!"),
            "Повідомлення для alice надіслано успішно"
        )
    def test_notify_failure(self):
        self.mock_service.send.return_value = False
        self.assertEqual(
            self.manager.notify_user("bob", "Привіт!"),
            "Не вдалося надіслати повідомлення для bob"
        )
    def test_send_called_with_correct_args(self):
        self.mock_service.send.return_value = True
        self.manager.notify_user("carol", "Тест")
        self.mock_service.send.assert_called_once_with("carol", "Тест")
```

```
def check_even(number): return number % 2 == 0
```

```
class TestCheckEven(unittest.TestCase):
    @parameterized.expand([
        ("zero", 0, True),
        ("two", 2, True),
        ("three", 3, False),
        ("minus_four", -4, True),
        ("minus_one", -1, False),
        ("one", 1, False),
        ("hundred", 100, True),
    ])
    def test_even(self, _, number, expected):
        self.assertEqual(check_even(number), expected)
```

```
if __name__ == "__main__":  
    unittest.main(argv=[""], exit=False)
```

Результат:

```
***  
.....  
-----  
Ran 16 tests in 0.020s  
  
OK  
<unittest.main.TestProgram at 0x783cae1cc440>
```

Висновок:

Під час виконання лабораторної роботи було опановано методи тестування програмного забезпечення за допомогою фреймворку **unittest**. Було реалізовано різні типи тестів: базові перевірки логіки, тестування обробки помилок, використання об'єктів-імітаторів (**Mock**) для ізоляції компонентів системи, а також параметризовані тести для перевірки функцій на великій кількості вхідних даних. Отримані результати підтвердили надійність розроблених класів та ефективність автоматизованого тестування.